

Round robin VS Priority Queue

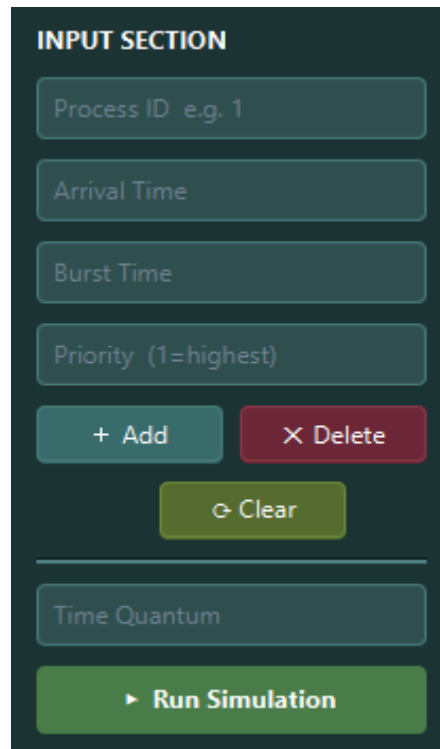
Names and IDs :

1.Nadeen Mohamed Farid	20245083 (Leader)
2.Hassan Ahmed Hassan	20245023
3.Amany Mohamed Ismael	20245012
4.Manar Mahmoud sayed	20245079 (Manar Selim in github)
5.Magda Ahmed Abo El-yazid	20245062
6.Mohamed Walid Abd El-Maged	20245073 (Saltawy in github)
7.Fadwa Hussin Ibrahim	20245057

Sections:

- 1.Input section (2~3)
- 2.Process table (4)
- 3.Results (5)
- 4.comparison (6)
- 5.App Final conclusion (7)
- 6.Test Scenarios (8~10)
- 7.Github Repository (10)

First, we have input section:



INPUT SECTION

Process ID e.g. 1

Arrival Time

Burst Time

Priority (1=highest)

+ Add X Delete

Clear

Time Quantum

▶ Run Simulation

1. User can Add process by entering all requirements

They should be Non-negative values and the highest priority is 10

Also no duplicated IDs

```
static void createProcess() { // usage: & HassanAhmed017 +1+
    System.out.println("Enter process ID: ");
    int processID = in.nextInt();

    System.out.println("Enter process Arrival Time: ");
    int processArrivalTime = in.nextInt();

    System.out.println("Enter process burst Time: ");
    int processburstTime = in.nextInt();

    System.out.println("Enter process Priority: ");
    int processPriority = in.nextInt();

    Process newP = new Process(processID, processArrivalTime, processburstTime, processPriority);
    try {

        validateProcess(newP, processes);

        newP.setResponseTime(-1);
        processes.add(newP);

        System.out.println("Process added successfully.");
    } catch (IllegalArgumentException e) {

        System.out.println("Validation Error: " + e.getMessage());
    }
}
```

2. User can delete a process by selecting it from process table then press delete

```
public static void removeProcess(int id) { 2 usages  👤 HassanAhmed017
    for (int i = 0; i < processes.size(); i++) {
        if (processes.get(i).getPid() == id) {
            processes.remove(i); // remove directly by index
            break;
        }
    }
}
```

3. User can clear process table by clicking Clear button

```
public static void Clear() { 6 usages  👤 HassanAhmed017
    processes.clear();
}
```

It clears process ArrayList and all the previous results

4. user can enter a Quantum and It should be non-negative value

```
static void enterQuantum() { 1 usage  👤 HassanAhmed017
    int temp = in.nextInt();
    validateQuantum(temp);
    quantum = temp;
}
```

2.Process Table:

PROCESS TABLE				
PID	Arrival	Burst	Priority	
1	0	5	3	
2	1	3	1	
3	2	8	2	
4	3	2	4	
5	4	6	2	

Here are all the entered processes and their Arrival time, BT and Priority

Ready Queue — Round Robin
Queue: [] — simulation complete

Ready Queue — Priority Scheduling
Queue: [] — simulation complete

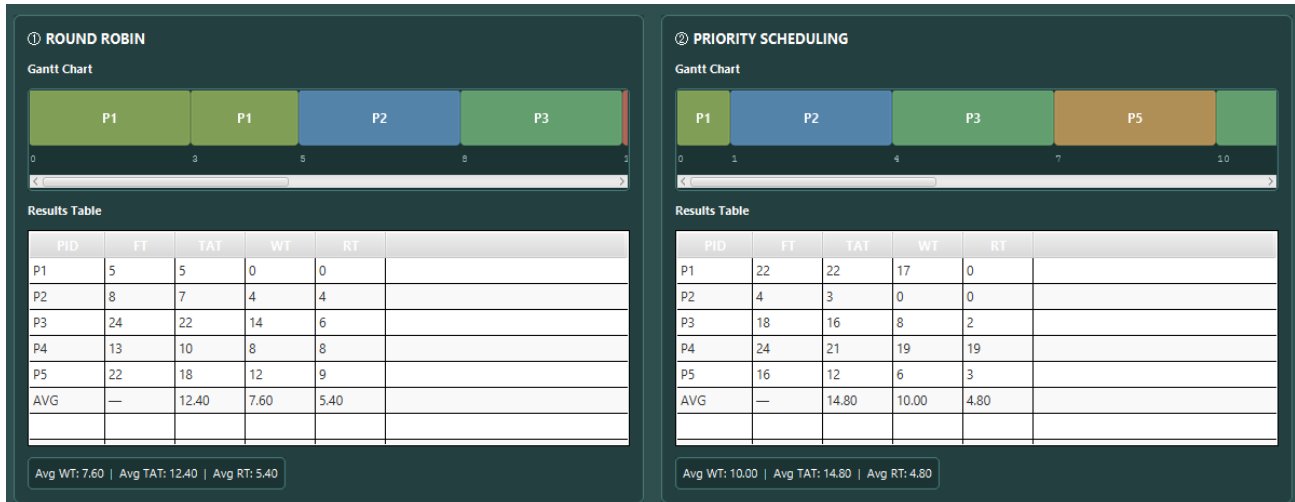
This is the ready Queue block, it looks like this when simulation ends

Ready Queue — Round Robin
Queue: [Running: P1 t=0→3 | Waiting: P1, P2, P3, ...]

Ready Queue — Priority Scheduling
Queue: [Running: P1 t=0→1 | Waiting: P2, P3, P5, ...]

While running

3.Results:



Here are all the results of the processes of PQ and RR.

FT: finish time of each process

TAT: Turnaround Time (time process took since it entered ReadyQueue)

WT: waiting time (process in RQ but not being executed)

RT: response time (First time the process entered RQ)

RESULT

Winner

Round Robin

Here the App Displays which process was better.

4.Comparison

📊 COMPARISON SUMMARY		
Metric	Round Robin	Priority
Avg Waiting Time	7.60	10.00
Avg Turnaround Time	12.40	14.80
Avg Response Time	5.40	4.80
Starvation Risk	None ✓	Possible ⚠
Fairness	High ✓	Low ✗
Best For	Interactive Systems	Batch / Priority Jobs

In this section the app shows which process was better in what, if there was any starvation risk, if there was fairness or no and finally what this Scheduling Algorithm better for

5.Final Conclusion:

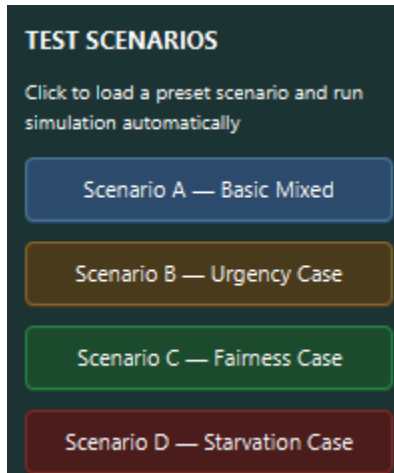
④ FINAL CONCLUSION
Q: Which algorithm had the shorter average waiting time? A: Round Robin kept processes waiting less on average — 7.60 vs 10.00 units.
Q: Which algorithm completed processes faster overall? A: Round Robin achieved a lower average turnaround time (12.40 vs 14.80 units), meaning processes spent less total time in the system.
Q: Which algorithm responded to processes more quickly? A: Priority Scheduling gave processes their first CPU slot sooner on average (4.80 vs 5.40 units, a negligible difference).
Q: Did high-priority processes get a real advantage? A: Yes — Priority Scheduling intentionally serves urgent processes first, so they finish much sooner than they would under Round Robin's fixed time-slice approach.
Q: Was Round Robin fair to all processes? A: Yes — every process received CPU time in rotation, regardless of priority. No process was skipped or delayed because of a lower priority number.
Q: Could any process starve under Priority Scheduling? A: Potentially yes — if high-priority processes keep arriving, a low-priority process may wait indefinitely. Round Robin avoids this by guaranteeing every process gets a turn.

These are some Q&A to determine whether RR is better or RQ

Written Conclusion
CONCLUSION Round Robin came out ahead in this simulation, winning 2 out of 3 measured metrics (waiting time, turnaround time, and response time). Round Robin reduced average waiting time by 2.40 units compared to Priority Scheduling. Round Robin finished processes 2.40 units sooner on average — a meaningful improvement in throughput for this workload. From a fairness perspective, Round Robin guaranteed every process received CPU time regularly, making it the safer choice when no process should be left waiting indefinitely. Priority Scheduling, while faster for urgent tasks, introduces the risk of starvation for lower-priority processes if the workload is not carefully balanced.

This is a written conc with more details

6. Test Scenarios



Each button clears RQ then loads it with different processes to test:

1. Normal workload (Basic mixed):

Loads 5 processes with varied arrival times, burst times, and priorities.

Serves as the baseline comparison between Round Robin and Priority Scheduling.

Round Robin distributes CPU time evenly; Priority Scheduling favors lower priority numbers.

Expected: RR wins on fairness; PQ may win on waiting time for high-priority processes.

```
public static SimulationResult loadScenarioA() { 1 usage  HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, burstTime: 5, priority: 3));
    processes.add(new Process( pid: 2, arrivalTime: 1, burstTime: 3, priority: 1));
    processes.add(new Process( pid: 3, arrivalTime: 2, burstTime: 8, priority: 2));
    processes.add(new Process( pid: 4, arrivalTime: 3, burstTime: 2, priority: 4));
    processes.add(new Process( pid: 5, arrivalTime: 4, burstTime: 6, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

2.Urgency case:

Loads 5 processes where P3 and P4 have priority=1 (highest urgency).

Tests whether Priority Scheduling visibly serves urgent processes before others.

Under PQ, P3 and P4 finish significantly earlier than under Round Robin.

Expected: PQ clearly outperforms RR in response time for high-priority processes.

```
public static SimulationResult loadScenarioB() { 1 usage  HassanAhmed017 *
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, burstTime: 10, priority: 4));
    processes.add(new Process( pid: 2, arrivalTime: 1, burstTime: 8, priority: 4));
    processes.add(new Process( pid: 3, arrivalTime: 2, burstTime: 4, priority: 1));
    processes.add(new Process( pid: 4, arrivalTime: 3, burstTime: 6, priority: 1));
    processes.add(new Process( pid: 5, arrivalTime: 4, burstTime: 3, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

3.Fairness case:

Loads 4 processes all arriving at t=0 with equal burst time (9) and equal priority (2).

Tests whether Round Robin distributes CPU time evenly across identical processes.

Under RR all processes receive equal time slices and finish at similar times; PQ is arbitrary.

Expected: RR shows perfect fairness; PQ finishes them sequentially with no fairness guarantee.

```
public static SimulationResult loadScenarioC() { //is RR fair? 1 usage  HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, burstTime: 9, priority: 2));
    processes.add(new Process( pid: 2, arrivalTime: 0, burstTime: 9, priority: 2));
    processes.add(new Process( pid: 3, arrivalTime: 0, burstTime: 9, priority: 2));
    processes.add(new Process( pid: 4, arrivalTime: 0, burstTime: 9, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

4.Starvation case:

Loads 4 high-priority processes (priority=1) and one low-priority process P5 (priority=4).

Tests whether a low-priority process suffers significantly longer waiting under Priority Scheduling.

Under PQ, P5 waits until all 4 higher-priority processes complete before receiving any CPU time.

Expected: P5 has near-zero wait under RR but maximum wait under PQ, demonstrating starvation risk.

```
public static SimulationResult loadScenarioD() {//low priority waits more 1usage HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 2, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 3, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 4, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 5, arrivalTime: 0, brustTime: 10, priority: 4));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

7.For more Details and code: [Git-Hub repository](#)